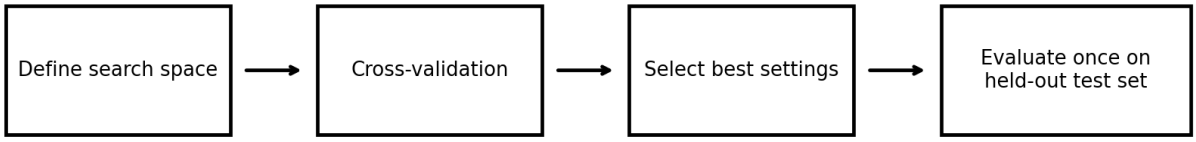
DAY 35

Hyperparameter Tuning and Model Optimization

A practical guide to GridSearchCV, RandomizedSearchCV, cross-validation, and leakage-safe model selection



Keep the test set separate from all tuning decisions

Learning goal: search for model settings systematically, compare candidates with cross-validation, and reserve the test set for a final unbiased check.

60-Day Data Science Learning Journey

1. What is hyperparameter tuning?

A machine-learning model learns patterns from training data. Hyperparameters are settings chosen before or around training that influence how the model learns. Hyperparameter tuning compares candidate settings using a defined validation strategy and scoring metric.

Parameters vs. hyperparameters

Parameters	Hyperparameters
Learned from training data, such as the coefficients in linear regression or split rules in a fitted tree.	Set by the practitioner or search procedure, such as tree depth, number of trees, or minimum samples for a split.

Common Random Forest hyperparameters

• **n_estimators:** number of trees. More trees can stabilize predictions but require more compute.

• **max_depth:** maximum tree depth. A restriction can reduce overfitting but may underfit.

• **min_samples_split:** minimum number of samples required to split a node. Higher values make splitting more conservative.

• **min_samples_leaf:** minimum samples allowed in a leaf. Can smooth predictions.

• **max_features:** number or proportion of features considered at a split. It affects tree diversity and performance.

• **random_state:** helps make randomized procedures reproducible. It is not a setting to optimize for performance.

2. GridSearchCV

GridSearchCV evaluates every combination specified in a parameter grid. It usually uses cross-validation on the training data to estimate how well each combination generalizes.

Example: 3 values for n_estimators × 3 values for max_depth × 2 values for min_samples_split = 18 combinations. With 5-fold CV, that means 18 × 5 = 90 fits, followed by a refit of the best candidate on all training data when refit=True.

```
param_grid = {
    "n_estimators": [100, 200, 300],
    "max_depth": [None, 5, 10],
    "min_samples_split": [2, 5],
}
search = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid=param_grid,
    scoring="f1",
    cv=5,
    n_jobs=-1,
    refit=True,
)
search.fit(X_train, y_train)
print(search.best_params_)
print(search.best_score_)
best_model = search.best_estimator_
```

Read the results: best_params_ gives the selected settings. best_score_ gives the best mean cross-validation score under the chosen metric. best_estimator_ is the refitted estimator using those settings.

3. RandomizedSearchCV

RandomizedSearchCV samples a fixed number of candidate combinations from lists or statistical distributions. It is useful when the search space is large or continuous, because it can explore more distinct values without evaluating every combination.

```
from scipy.stats import randint
```

```
param_distributions = {
    "n_estimators": randint(100, 501),
    "max_depth": [None, 5, 10, 15, 20, 30],
    "min_samples_split": randint(2, 11),
    "min_samples_leaf": randint(1, 6),
}
random_search = RandomizedSearchCV(
    RandomForestClassifier(random_state=42),
    param_distributions=param_distributions,
    n_iter=20,
    scoring="f1",
    cv=5,
    random_state=42,
    n_jobs=-1,
)
random_search.fit(X_train, y_train)
```

4. Grid search vs. randomized search

GridSearchCV	RandomizedSearchCV
Tests every combination in the provided grid.	Tests a fixed number of sampled combinations.
Best for small, carefully chosen grids.	Best for broad or large search spaces.
Compute cost grows quickly as grid size increases.	Compute can be controlled with n_iter.
All specified values are evaluated.	Some possible combinations may never be sampled.

5. A safe end-to-end workflow

1. Split data into training and test sets before tuning. Use stratify=y for classification when appropriate.
2. Choose a metric that reflects the real objective. Accuracy can be misleading on imbalanced classes.
3. Define a reasonable search space based on model behavior and compute limits.
4. Use cross-validation on training data to compare candidates.
5. Select the best candidate from CV, then evaluate it on the untouched test set once.
6. Report the metric, confusion matrix, classification report, and relevant limitations.

Why leakage matters

Data leakage happens when information that would not legitimately be available at training time influences training or model selection. Do not tune hyperparameters against the final test score. If preprocessing learns from data, fit it inside a scikit-learn Pipeline so each CV fold learns preprocessing only from that fold's training portion. The test set is not a dashboard for repeated experimentation.

6. Choosing the scoring metric

Metric	Useful when
Accuracy	Overall fraction of correct predictions, most informative when class balance and error costs make it appropriate.
Precision	False positives are costly and predicted positives should be trustworthy.
Recall	Missing positive cases is costly.
F1-score	A balance of precision and recall is desired.
ROC-AUC	Ranking ability across classification thresholds is useful. Also consider PR-AUC for highly imbalanced data.

7. Complete Python practice walkthrough

The companion .py file runs both GridSearchCV and RandomizedSearchCV on the same training split. It prints selected settings, mean CV F1, holdout metrics, a confusion matrix, and a classification report. Install scikit-learn and scipy if they are missing.

```
pip install scikit-learn scipy
python Day-35-Python-Practice.py
```

Dataset note: the script uses the built-in Breast Cancer Wisconsin dataset for educational classification practice only. It is not a medical tool and must not be used to diagnose anyone.

8. Common mistakes

• Optimizing accuracy automatically, even when precision or recall matters more.

• Searching an enormous grid without estimating the number of fits and runtime.

• Looking at the test score repeatedly and adjusting the search to improve it.

• Using the same cross-validation folds or data for too many iterative decisions without acknowledging selection bias.

• Forgetting stratification for classification tasks where class proportions matter.

• Assuming a higher cross-validation score guarantees better real-world performance. Results vary with data, sampling, and deployment conditions.

• Changing random_state until a pleasing test score appears. This is not valid optimization.

9. Practice exercises with solutions

Exercise 1. A grid contains 4 values for n_estimators, 3 for max_depth, and 2 for min_samples_split. How many combinations and fits with 5-fold CV?

Solution: 4 × 3 × 2 = 24 combinations. With five folds, 24 × 5 = 120 CV fits, plus a final refit if enabled.

Exercise 2. When might RandomizedSearchCV be preferable?

Solution: When there are many possible settings, numeric ranges are broad, or compute is limited. n_iter lets you cap the number of candidates.

Exercise 3. Your task is to catch as many positive cases as possible. Which metric might be important?

Solution: Recall is a likely priority, but confirm error costs and the application context. Precision and threshold behavior should also be reported.

Exercise 4. Why should the test set remain untouched during tuning?

Solution: Repeated decisions based on test results leak information from the test set into model selection, making the final estimate overly optimistic.

10. Quick revision checklist

■ I can explain parameters vs. hyperparameters.

■ I can count grid combinations and CV fits.

■ I can compare GridSearchCV with RandomizedSearchCV.

■ I can explain best_params_, best_score_, and best_estimator_.

■ I can choose a scoring metric for the problem.

■ I keep preprocessing inside pipelines when it learns from data.

■ I tune using training data and CV, then evaluate on the held-out test set.

Day 35 takeaway

Hyperparameter tuning is a disciplined search, not a guarantee of a perfect model. Define a suitable search space, validate candidates fairly, use a metric that matches the objective, and keep the final test set separate.

Next step: run the practice script, read the printed outputs, and explain why the chosen metric and validation strategy suit this example.